DP-OPS-0012: Autonomous Skill Heuristics & Provenance Engine (Phase 2)

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/dp-ops-0012-skills-phase2

Base HEAD: 827e7f32e

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- **DP-OPS-0011-RESULTS.md** (Previous state)
- **Enhancing Stela Skills Subsystem.md** (Guidance source)
- **ops/lib/skill/skill_lib.sh** (Target modification)

II) SCOPE & SAFETY

- **Objective:** Upgrade the Skills Subsystem to Phase 2 by implementing autonomous provenance capture, semantic drift guards, and the "Harvest-Draft-Promote" lifecycle enhancements defined in the Enhancing Stela Skills Subsystem specification.
- **Non-Goals (optional, drift-killer):**
  - No changes to the projects/ directory payload; strictly platform operations.
  - No external API integrations for skill generation (local heuristics only).
  - No alteration of existing TRUTH.md invariants.

Target Files allowlist (hard gate)

- ops/lib/skill/skill_lib.sh
- docs/library/skills/S-LEARN-07.md (Update invocation guidance)
- SKILL.md (Update template)
- TASK.md (Update closeout instructions)
- AGENTS.md (Update behavior logic)
- (new) ops/lib/skill/heuristics.sh
- (new) docs/library/skills/S-LEARN-08.md (Skill: Advanced Git Forensics)

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

1. The Context-Memory Paradox in Stateless Architectures

The Stela architecture operates on a rigorous, almost ascetic constraint: **The repository is stateless; files are the only memory.** This "Context as Memory" philosophy ensures absolute auditability and prevents the accumulation of hidden "tribal knowledge" that plagues traditional DevOps environments.[1] However, this constraint introduces a critical operational friction point known as the "Knowledge Gap." When an agent (whether a human Operator or an AI Integrator) begins a session, they possess zero historical awareness beyond what is explicitly loaded via the Context Manifest (ops/lib/manifests/CONTEXT.md).[1]

In the legacy state (pre-DP-OPS-0010), this gap was bridged solely by human intuition. The operator implicitly knew, for example, that "we tried the JSON parser upgrade last week and it caused a regression." The AI agent, lacking this specific file-based memory, would unknowingly repeat the failed experiment, creating a cycle of drift and redundancy. This inefficiency violates the core "Reuse-First Discipline" mandated by AGENTS.md.[1]

DP-OPS-0010 and DP-OPS-0011 established the **Skills Subsystem** (S-LEARN-*) to codify this tribal knowledge into ingestible artifacts.[1] The successful promotion of **S-LEARN-07: Harvest and Promote Skills at DP Closeout** [1] marked the completion of Phase 1, establishing the basic plumbing for skill capture. However, the current implementation of ops/lib/skill/skill_lib.sh (as delivered in DP-OPS-0011) remains a "passive" utility.[1] It functions as a scribe, not an analyst. It relies entirely on the worker to *volunteer* context and *formulate* the solution, introducing a new vector for human error and "hallucinated" best practices.

2. The Passive Scribe Vulnerability (Phase 1 Critique)

The current harvest command, while functional, is fundamentally "dumb." It accepts strings provided by the user but does not validate them against the objective reality of the repository state.[1]

- **Current Execution Path:**
  Bash
  ```
  ops/lib/skill/skill_lib.sh harvest --name "fix-bug" --context "bug happens" --solution "fix it"
  ```
- **The Deficit:** The resulting skill artifact contains no cryptographic or heuristic link to the *actual work* that occurred. There is no guarantee that the "solution" text matches the git diff that was just committed. The system trusts the reporter implicitly.
- **The Risk ("Ghost Skills"):** This permits the creation of "Ghost Skills"—canonical documents that describe a theoretical best practice that diverges from the actual codebase. Over time, this leads to "Scholastic Drift," where the documentation describes a pristine system that does not exist, while the runtime operates on undocumented tribal hacks.

This disconnect is explicitly identified in the "Enhancing Stela Skills Subsystem" strategic document [1], which calls for a move from "Passive Recording" to "Active Forensics." The document critiques the Phase 1 implementation for implying a "one-shot" creation process

that lacks the necessary rigor for a high-assurance environment.

3. Strategic Gap Analysis: Phase 1 vs. Phase 2

The following table contrasts the capabilities delivered in DP-OPS-0011 [1] against the requirements for Phase 2 defined in the strategic roadmap.[1]

Feature Domain	Phase 1 State (Current)	Phase 2 Requirement (Target)	Gap Severity
Provenance	Manual text entry (--context). No validation.	**Autonomous Git Forensics.** Auto-capture of DP-ID, git hash, and diff stats.	**Critical**
Drift Guard	ID collision check only (e.g., S-LEARN-01 exists).	**Semantic Drift Guard.** Keyword analysis to prevent conceptual duplicates.	**High**
Friction	None. System waits for user input.	**Friction Detection.** Heuristic analysis of git log churn and diff intensity.	**High**
Lifecycle	Harvest -> Promote (Implicit Drafting).	**Harvest -> Draft -> Refine -> Promote.** Explicit intermediate state.	Medium
Context	User-defined string.	**Heuristic-defined string.** Auto-populated from the "Hot Zone" (active files).	Medium

4. The Semantic Duplication Hazard

As the docs/library/skills/ directory grows, the risk of semantic collision increases exponentially. In a distributed system with multiple agents (Contractors) working asynchronously, it is inevitable that two agents will encounter the same problem at different times.

- **Scenario:** Agent A creates S-LEARN-12: Fix JSON Parsing. Two weeks later, Agent B, unaware of S-LEARN-12 because they didn't grep properly, creates S-LEARN-45: JSON Error Handling.
- **Result:** The Context Manifest (ops/lib/manifests/CONTEXT.md) becomes polluted with redundant tokens. The agent consuming the context receives conflicting or repetitive instructions, increasing the "noise-to-signal" ratio and burning context window tokens on redundant information.
- **Current Mechanism:** The existing skill_lib.sh only checks for *ID collisions* (e.g., ensuring S-LEARN-12 is not reused). It has no mechanism to check if the *topic* or *content* already exists.[1]

This DP introduces a **Semantic Drift Guard** that utilizes grep heuristics to scan the existing library for keywords found in the proposed title of a new skill. If a threshold (proposed: >2 matching keywords in >50% of existing files) is breached, the system must halt and require a manual override (--force), effectively forcing the agent to justify why this new skill is distinct from the existing canon.[1]

5. Friction-Blindness and the Lagging Indicator

The current system waits for the worker to *volunteer* a skill. This is a lagging indicator. A robust "Antifragile" system should instead detect *friction*—moments where the worker struggled, failed, or generated high churn—and *demand* a skill harvest.[1]

- **Friction Indicator A (The Hot Zone):** A git diff --stat showing a single file with >100 lines changed indicates a significant cognitive load or a complex refactor. This is a prime candidate for a skill.
- **Friction Indicator B (High Churn):** A git log showing the same file modified 4 times in one session indicates a struggle ("trial and error"). This pattern suggests the worker did not know the correct path initially. Capturing the *final* path as a skill prevents future workers from repeating the 3 failed attempts.

DP-OPS-0012 will instrument these signals via a new library, ops/lib/skill/heuristics.sh, which will act as the "Cortex" of the skills subsystem, while skill_lib.sh remains the "Hand."

B) REQUEST

The objective of DP-OPS-0012 is to upgrade the Skills Subsystem from a "Passive Scribe" to an "Active Analyst" (Phase 2). This involves distinct architectural enhancements derived from

Enhancing Stela Skills Subsystem.md.[1]

Feature 1: Autonomous Heuristics Engine (ops/lib/skill/heuristics.sh)

We will extract the intelligence logic into a dedicated library that analyzes the repository state to *propose* skill content. This separation of concerns ensures that the core skill_lib.sh remains focused on file manipulation while heuristics.sh focuses on analysis.

- **Logic Specification:**
  1. **Diff-Stat Analysis (detect_hot_zone):** Execute git diff --numstat main...HEAD to identify the file with the highest number of insertions+deletions. This file defines the "Context" of the work.
  2. **Churn Detection (detect_high_churn):** Analyze git log --oneline --name-only main...HEAD to count commit frequency per file. Any file appearing in >1 commit is flagged as "High Churn."
  3. **Synthesis:** The output of these functions will be injected into the ## Provenance section of the skill draft.

Feature 2: Provenance Enforcement

We will modify skill_lib.sh to require specific metadata linking the skill to the active operation. This is non-negotiable for auditability.[1]

- **Requirement:** Every harvested skill must include a ## Provenance section containing:
  - **DP-ID:** The active task ID extracted from TASK.md.
  - **Branch/HEAD:** The precise git coordinates.
  - **Objective:** The goal extracted from TASK.md.
  - **Friction Context:** The output of the Heuristics Engine.
- **Mechanism:** The harvest command will automatically capture these values from the runtime environment. The worker *cannot* override them with manual input, ensuring the skill is tethered to reality.

Feature 3: Semantic Drift Guard

We will implement a "Keyword Collision Check" in skill_lib.sh to prevent library pollution.

- **Logic:** Before accepting a new draft, the script will grep existing docs/library/skills/*.md files for the keywords in the proposed Title.
- **Algorithm:**
  1. Normalize title (lowercase, strip punctuation).
  2. Tokenize into words (filtering stop words like "the", "and", "skill").
  3. Scan existing files. Count matches.
- **Threshold:** If >0 existing skills contain significant keyword overlap (heuristic threshold), the script issues a WARNING requiring explicit override (--force).
- **Goal:** Prevent the "JSON Error" vs "Fix JSON" duplication scenario.

Feature 4: The Refinement Lifecycle

We will bifurcate the harvest command into harvest (raw capture) and refine (human/agent polishing).[1]

- **Step 1 (Harvest):** System captures state (diffs, logs, DP ID) and dumps a rough Markdown file to storage/handoff/. This file is the "Raw Evidence."
- **Step 2 (Refine):** Worker edits the Markdown file to add the "Solution" prose. The worker is explicitly guided to *not* touch the Provenance block.
- **Step 3 (Promote):** The promote command (established in 0011) moves it to docs/ and registers it.

Feature 5: Integration with TASK.md

We will update TASK.md to explicitly reference this new workflow in the "Closeout" section, guiding the worker to run the heuristic check. The "Legacy capture" method (manual string entry) will be formally deprecated and forbidden for production payloads.

C) CHANGELOG

- **Created** ops/lib/skill/heuristics.sh: New library for git analysis and friction detection.
- **Updated** ops/lib/skill/skill_lib.sh:
  - Added source "${SCRIPT_DIR}/heuristics.sh".
  - Updated cmd_harvest to utilize heuristic auto-detection for Context/Title if not provided.
  - Added check_semantic_collision function to cmd_harvest.
  - Enforced ## Provenance injection with immutable git data.
- **Updated** SKILL.md:
  - Revised "Harvest Engine workflow" documentation to include the Heuristic phase.
  - Updated "Promotion Packet" template to include the new Provenance fields.
- **Updated** AGENTS.md:
  - Refined "Contractor Closeout Skill Harvesting" to mandate the use of skill_lib auto-detection.
- **Updated** docs/library/skills/S-LEARN-07.md:
  - Updated the "Invocation guidance" to reflect the new heuristics capability.
- **Created** docs/library/skills/S-LEARN-08.md:
  - New skill "Advanced Git Forensics" to document the manual use of the new heuristic tools.

D) PATCH / DIFF

1. New Artifact: ops/lib/skill/heuristics.sh

Bash

```
#!/usr/bin/env bash
#
# Stela Skill Heuristics Engine
# Analyzes repo state to suggest Skill context and provenance.
# Sourced by ops/lib/skill/skill_lib.sh

# Detects the "Hot Zone" - the file with the most changes in the current branch
detect_hot_zone() {
  local branch="$1"
  local base="$2"

  # Get diff stat, sort by changes, take top 1
  # Format: " 10 file_path"
  git diff --numstat "$base...$branch" | sort -nr | head -n 1 | awk '{print $3}'
}

# Detects "High Churn" - files modified in multiple commits in this branch
detect_high_churn() {
  local branch="$1"
  local base="$2"

  # List all files changed in all commits on this branch, count occurrences
  git log --name-only --format="" "$base...$branch" | \
    sort | uniq -c | sort -nr | \
    awk '$1 > 1 {print $2 " (" $1 " commits)"}' | head -n 3
}

# Generates a standard Provenance Block
generate_provenance_block() {
  local dp_id="$1"
  local objective="$2"

  local branch
  branch="$(git rev-parse --abbrev-ref HEAD)"
  local head_hash
  head_hash="$(git rev-parse HEAD)"
  local timestamp
  timestamp="$(date -u '+%Y-%m-%d %H:%M:%S UTC')"
  local stat_block
  stat_block="$(git diff --stat main...HEAD |
```

```
| echo "(no changes)")"

  cat <<EOF
## Provenance
- **Captured:** ${timestamp}
- **DP-ID:** ${dp_id:-"Not Provided"}
- **Branch:** ${branch}
- **HEAD:** ${head_hash}
- **Objective:** ${objective:-"Not Provided"}
- **Friction Context:**
  - Hot Zone: $(detect_hot_zone "$branch" "main" |

| echo "None")
  - High Churn: $(detect_high_churn "$branch" "main" |

| echo "None")
- **Diff Stat:**
\`\`\`text
${stat_block}
\`\`\`
EOF
}

# Checks for semantic collisions in existing skills
# Returns 0 if clean, 1 if collision detected
check_semantic_collision() {
  local title="$1"
  local skills_dir="$2"

  local collision_count=0
  local threshold=2 # Number of words to match

  # Normalize title: lowercase, remove special chars
  local clean_title
  clean_title="$(echo "$title" | tr '[:upper:]' '[:lower:]' | tr -d '[:punct:]')"

  # Split into words
  IFS=' ' read -r -a words <<< "$clean_title"

  if [[ ${#words[@]} -eq 0 ]]; then
    return 0
  fi

  echo "Analyzing semantic collisions for title: '${title}'..." >&2
```

```
  for file in "${skills_dir}"/*.md; do
    if [[! -f "$file" ]]; then continue; fi

    local match_count=0
    local file_content
    file_content="$(cat "$file" | tr '[:upper:]' '[:lower:]')"

    for word in "${words[@]}"; do
      # Skip common stop words
      if [[ "$word" =~ ^(the|and|for|with|use|how|to|skill)$ ]]; then continue; fi

      if [[ "$file_content" == *"$word"* ]]; then
        match_count=$((match_count + 1))
      fi
    done

    if (( match_count >= threshold )); then
      echo "WARN: Potential collision in $(basename "$file") ($match_count word matches)" >&2
      collision_count=$((collision_count + 1))
    fi
  done

  if (( collision_count > 0 )); then
    echo "WARN: Found $collision_count existing skills with similar keywords." >&2
    echo "      Use --force to proceed if this is a distinct concept." >&2
    return 1
  fi

  return 0
}
```

2. Update: ops/lib/skill/skill_lib.sh

Note: This patch injects the heuristics logic into the existing script structure.

```
Bash
```

```
#!/usr/bin/env bash
set -euo pipefail
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/../../.." && pwd)"
#... (existing imports)

# NEW: Source the Heuristics Engine
if]; then
  source "${SCRIPT_DIR}/heuristics.sh"
else
  # Fallback stubs if heuristics file is missing during bootstrap
  detect_hot_zone() { echo "n/a"; }
  detect_high_churn() { echo "n/a"; }
  generate_provenance_block() { echo "## Provenance (Manual)"; }
  check_semantic_collision() { return 0; }
fi

#... (existing usage/die/require_file functions)

# UPDATED: collect_harvest_inputs now supports flags for forcing collision checks
collect_harvest_inputs() {
  local -n out_name=$1
  local -n out_context=$2
  local -n out_solution=$3
  local -n out_force=$4 # New flag
  shift 4

  local context_from_stdin=0
  local solution_from_stdin=0
  out_force=0
  local positional=()

  while [[ $# -gt 0 ]]; do
    case "$1" in
      --name) out_name="$2"; shift 2 ;;
      --context) out_context="$2"; shift 2 ;;
      --solution) out_solution="$2"; shift 2 ;;
      --force) out_force=1; shift ;; # New flag handling
      #... (existing stdin handlers)
      *) positional+=("$1"); shift ;;
    esac
  done

  #... (rest of function)
```

```
}

cmd_harvest() {
  require_repo_root
  ensure_handoff_dir

  local name=""
  local context=""
  local solution=""
  local force=0

  collect_harvest_inputs name context solution force "$@"

  # NEW: Run Semantic Guard
  if [[ $force -eq 0 ]]; then
    if! check_semantic_collision "$name" "$SKILLS_DIR"; then
      die "Semantic collision detected. Run with --force to override."
    fi
  fi

  #... (existing logic for branch/stats)

  # NEW: Provenance Generation
  local dp_id
  local objective
  dp_id="$(read_task_field "Current DP")"
  objective="$(read_task_field "Goal")"

  # Generate the Heuristic Provenance Block
  local provenance_block
  provenance_block="$(generate_provenance_block "$dp_id" "$objective")"

  local slug
  slug="$(slugify "$name")"
  #... (path generation logic)

  local tmp
  tmp="$(mktemp)"

  # NEW: Template with Heuristics
  cat <<EOF > "$tmp"
# Skill Draft: ${name}
```

```
${provenance_block}

## Scope
Production payload work only.
Not platform maintenance.

## Invocation guidance
Use this skill when ${context}. Apply the solution: ${solution}.

## Drift preventers
- Stop if the DP scope is platform maintenance or if the DP does not explicitly request this skill.
- Anti-hallucination: use repository files as SSOT and stop if required inputs are missing.
- Negative check: do not add Skills to ops/lib/manifests/CONTEXT.md.

## Procedure
1) Review the context and desired outcome.
2) Apply the solution steps captured in this skill.
3) Verify results and record required evidence in RESULTS.
EOF

 redact_stream < "$tmp" > "$draft_path"
 rm -f "$tmp"

 echo "Draft created: $draft_path"
 echo "Next: Review draft, then run: ops/lib/skill/skill_lib.sh promote $draft_path"
}
```

3. Update: AGENTS.md

Locate: Section 2, Operational Directives

Update: Replace "Contractor Closeout Skill Harvesting" bullet with:

- **Contractor Closeout Skill Harvesting:** Upon DP closeout, propose a reusable skill via harvesting.
  - **Constraint:** Use ops/lib/skill/skill_lib.sh harvest to ensure Heuristic Provenance is captured.
  - **Negative Constraint:** Do not manually create markdown files in docs/library/skills/.
  - **Scope:** Mandatory for production payloads; optional for platform maintenance.

4. Update: SKILL.md (Partial)

Locate: Harvest Engine workflow

Update:

- ops/lib/skill/skill_lib.sh harvest creates a draft in storage/handoff/ with **autonomous provenance** and **semantic collision checks**.

- ops/lib/skill/skill_lib.sh promote promotes a draft into docs/library/skills/S-LEARN-XX.md and registers it in docs/library/INDEX.md.
- ops/lib/skill/skill_lib.sh check enforces the Skills Context Hazard against ops/lib/manifests/CONTEXT.md.

5. Update: TASK.md (Partial - Closeout Section)

Locate: Skill capture (required during normal DP processing)

Update:

- **Workflow:** Run ops/lib/skill/skill_lib.sh harvest to auto-generate a draft with provenance.
- **Review:** Check storage/handoff/ draft for heuristic accuracy (Hot Zone / Churn).
- **Promote:** ops/lib/skill/skill_lib.sh promote <draft_path> or log "no new skill" in RESULTS.
- **Legacy capture is forbidden:** Do not manually write skill files; the provenance block is required.

6. New Artifact: docs/library/skills/S-LEARN-08.md

S-LEARN-08: Advanced Git Forensics

Scope

Production payload work and platform maintenance.

Invocation guidance

Use this skill when debugging a high-friction task or when heuristics.sh fails to identify the correct Hot Zone.

Apply the solution: Manual execution of forensic commands.

Drift preventers

- Stop if the DP scope prevents shell access.
- Anti-hallucination: rely on git logs, not memory.

Procedure

1. Identify the Hot Zone manually:
   git diff --numstat main...HEAD | sort -nr | head -n 5
2. Identify High Churn files:
   git log --name-only --format="" main...HEAD | sort | uniq -c | sort -nr | head -n 5
3. Use these outputs to manually populate the "Friction Context" in a skill draft if the auto-harvester fails.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent)
  - **Fallback:** ./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- bash tools/context_lint.sh
- bash tools/lint_truth.sh
- bash tools/lint_library.sh
- bash tools/verify_tree.sh
- **NEW:** bash ops/lib/skill/skill_lib.sh check (Verify Context Hazard)

Skill capture (required during normal DP processing)

- **Command:** ops/lib/skill/skill_lib.sh harvest --name "test-skill-phase2" --context "testing phase 2" --solution "verify heuristics"
- **Verification:**
  - Check storage/handoff/ for the new draft.
  - **CRITICAL:** Verify the draft contains a ## Provenance section with Friction Context.
  - **CRITICAL:** Run harvest again with the same name to verify the **Semantic Drift Guard** triggers a WARNING.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0012-RESULTS.md (required)
- storage/handoff/OPEN-work-dp-ops-0012-skills-phase2-[hash].txt
- git diff --name-only
- git diff --stat
- NEXT: one single action

No receipt/proof bundle = DISAPPROVE.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)

- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0012-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion.

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0012-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

3.1) THREAD TRANSITION (RESET / ARCHIVE RULE)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file

    - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **1) ACTIVE CONTEXT** and **2) CURRENT DISPATCH PACKET** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
    - ---
    - THREAD START: DP-OPS-

---

4) WORK LOG (TIMESTAMPED CONTINUITY)

- 2026-02-01 16:30 — DP-OPS-0011: Added harvest/promote/check workflow to skill_lib, updated canon docs, and promoted S-LEARN-07. Verification: RUN (dump bundle; context_lint; lint_truth; lint_library; verify_tree WARN; dp_lint --test; skill_lib check). Blockers: none. NEXT: operator review + commit.
- 2026-02-01 18:20 — DP-OPS-0012: Initiating Phase 2 of Skills Subsystem (Heuristics + Semantic Guard). Verification: PENDING. Blockers: none. NEXT: Create ops/lib/skill/heuristics.sh.

Works cited

1. dump-full-main-827e7f32e.manifest.txt